



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



Machine Learning - 216H03C601

Dr. Nirmala Balloorkar

Assistant Professor

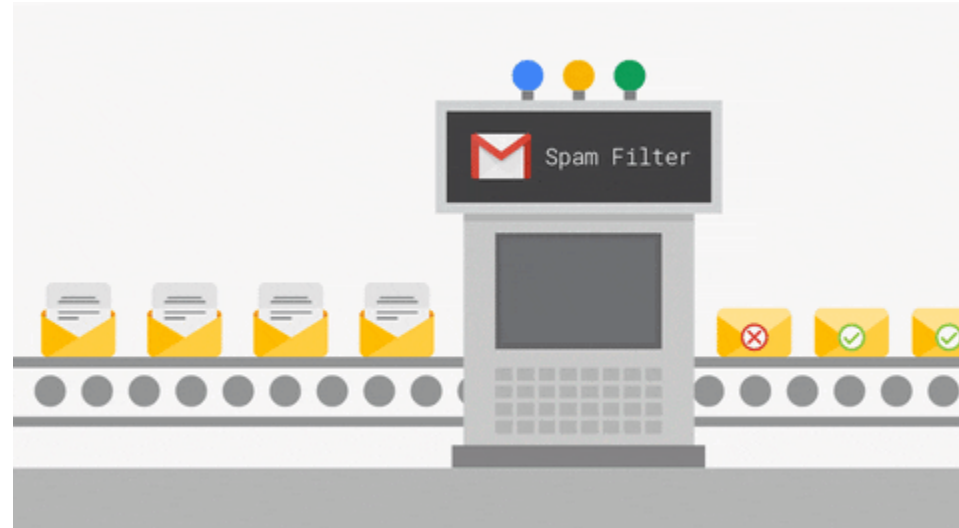
Department of Computer Engineering KJSSE

Traditional Programming

Traditional Programming



Email Spam/Non-spam classification

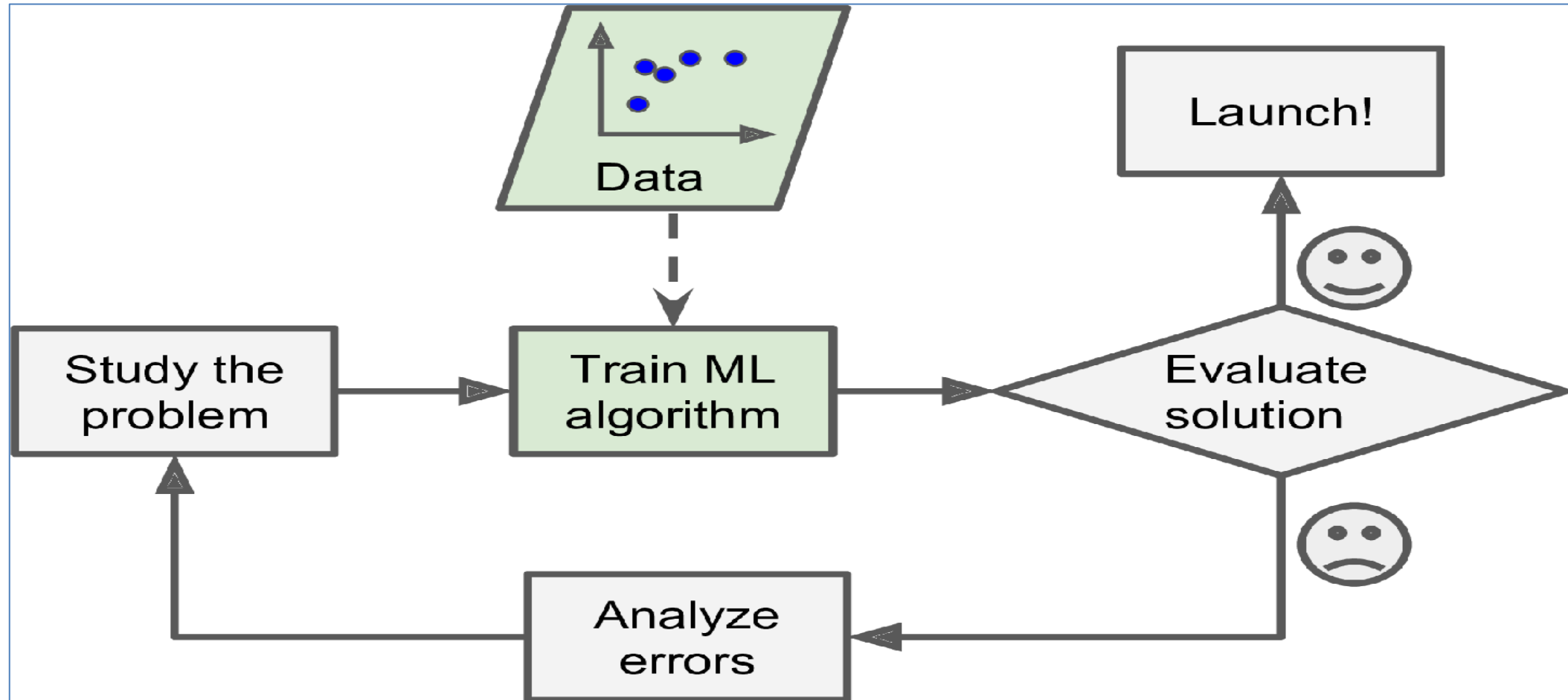


A spam filter built with traditional programming might use a list of banned words. The programmer writes rules: "If an email contains 'win money', mark as spam." Every new spam pattern requires a new rule.

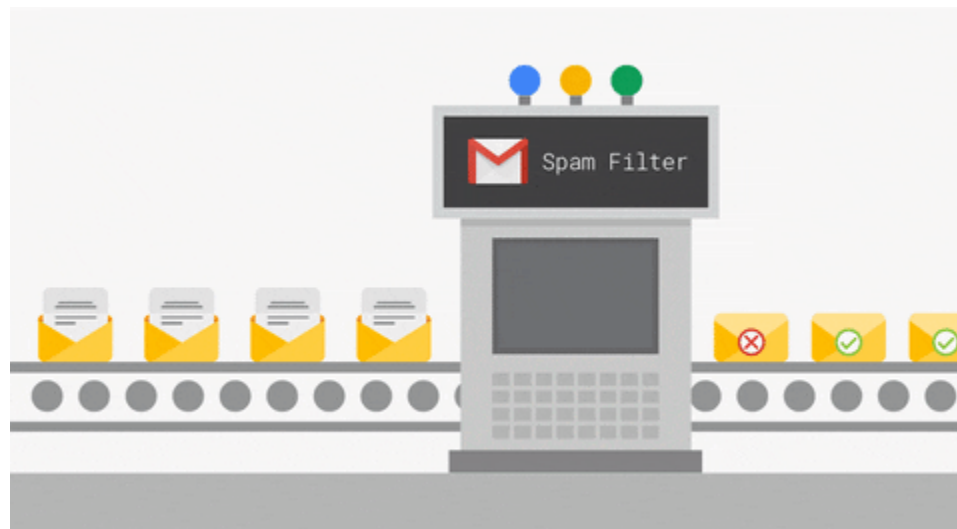
Characteristics

- Rule-Based Logic:
 - The programmer defines every condition and outcome.
- Deterministic Output:
 - The same input always produces the same result.
- Manual Updates:
 - Any change in requirements or data format requires manual code changes.
- Transparency:
 - The logic is clear, traceable, and easy to debug.
- Limited Flexibility:
 - Struggles with complex or unpredictable scenarios.

Machine Learning Approach



Email Spam/Non-spam classification



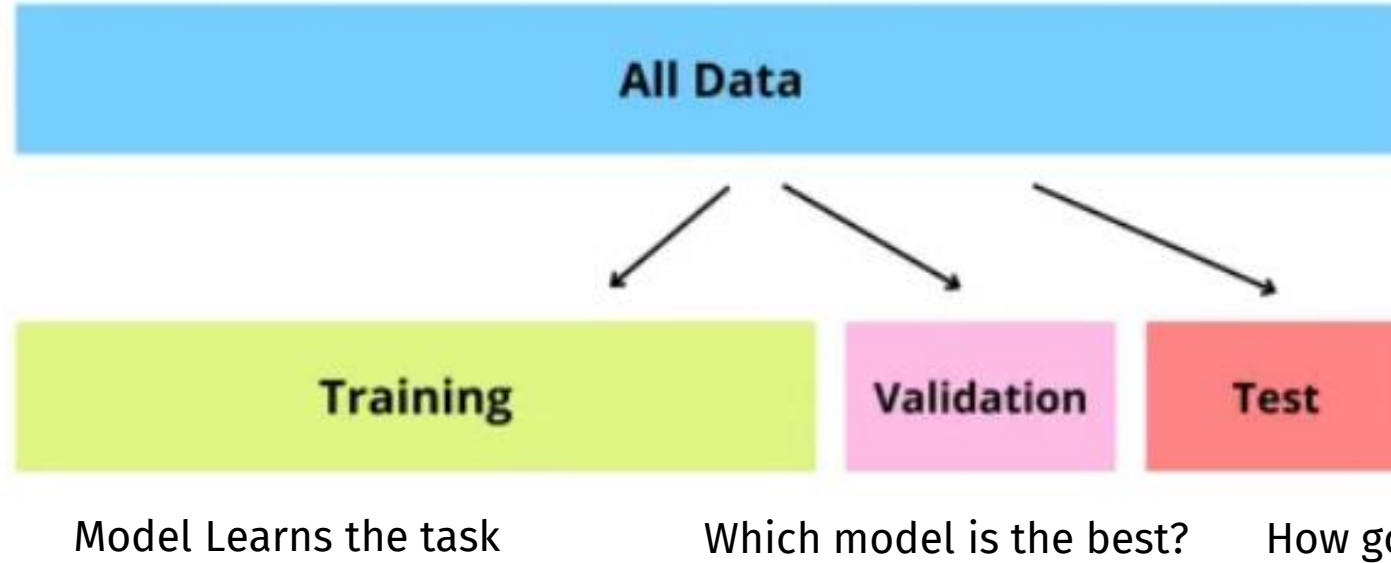
A spam filter built with machine learning is trained on thousands of emails labeled as "spam" or "not spam."

The model learns patterns in the data (word combinations, sender behavior, etc.) and can identify new spam emails even if they don't match any pre-written rule.

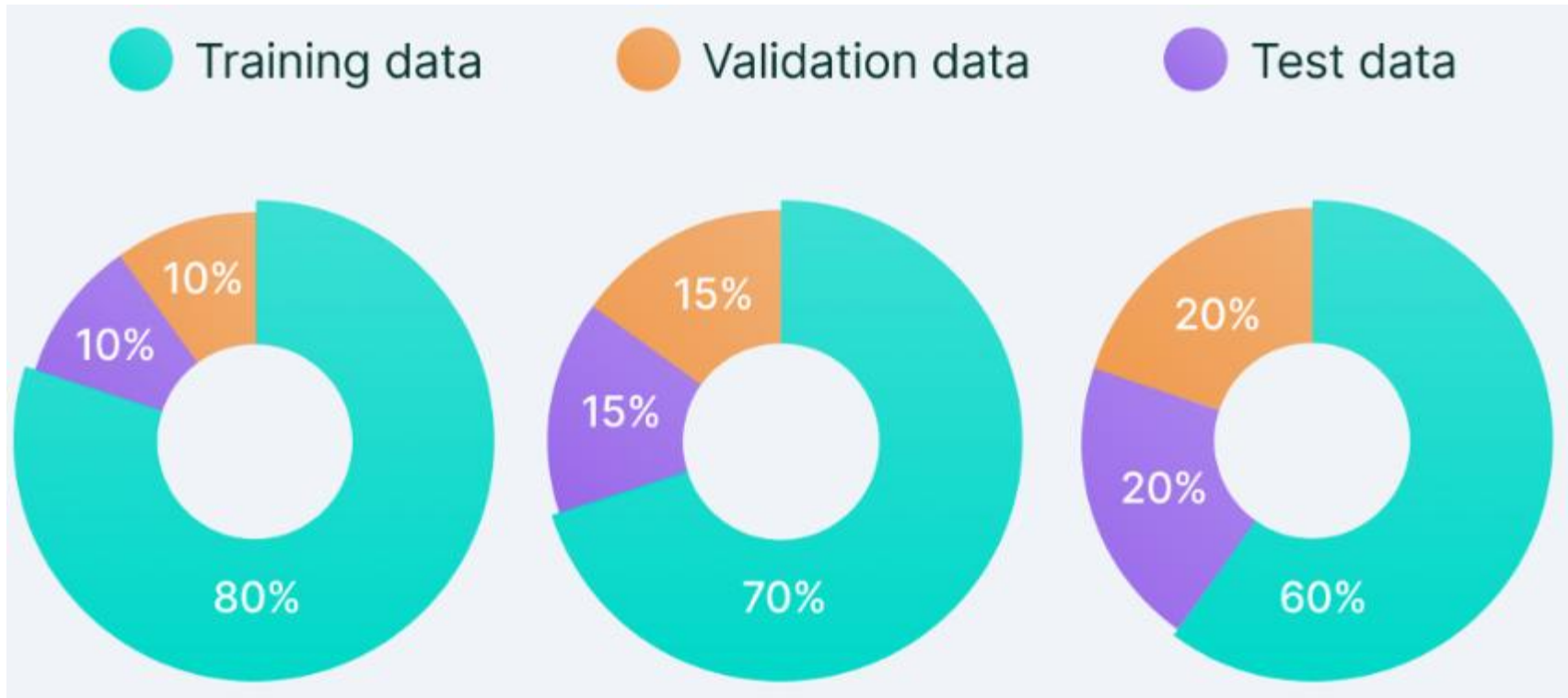
Characteristics

- Data-Driven Learning:
 - The model learns patterns from examples, not from explicit rules.
- Probabilistic Output:
 - Results are predictions with associated probabilities, not fixed answers.
- Adaptability :
 - The system can improve and adapt as new data arrives, often without manual intervention.

Key Concept



How to split machine learning data



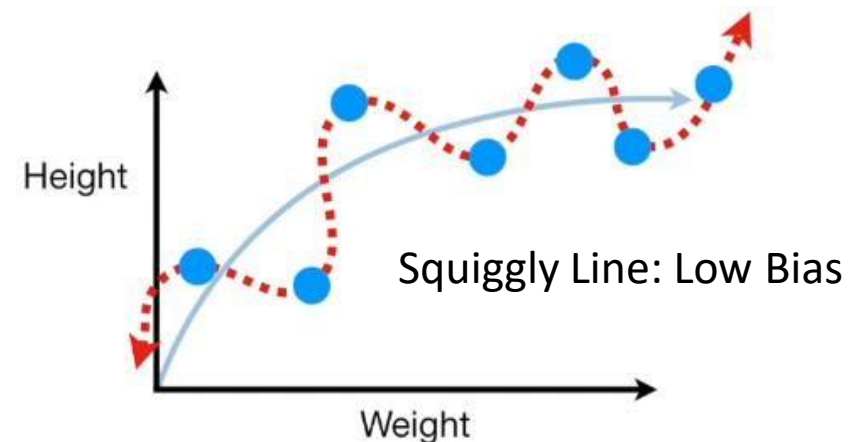
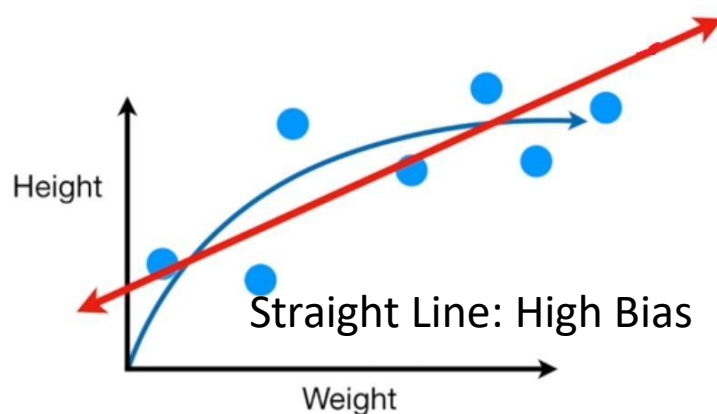
Evaluating your Machine Learning Model

A supervised Machine Learning model aims to train itself on the input variables(X) in such a way that the predicted values(Y) are as close to the actual values as possible. This difference between the actual values and predicted values is the **error** and it is used to evaluate the model. The error for any supervised Machine Learning algorithm comprises of 3 parts:

1. **Bias error (reducible errors)**
2. **Variance error (reducible errors)**
3. **The noise (irreducible error)**

What is Bias?

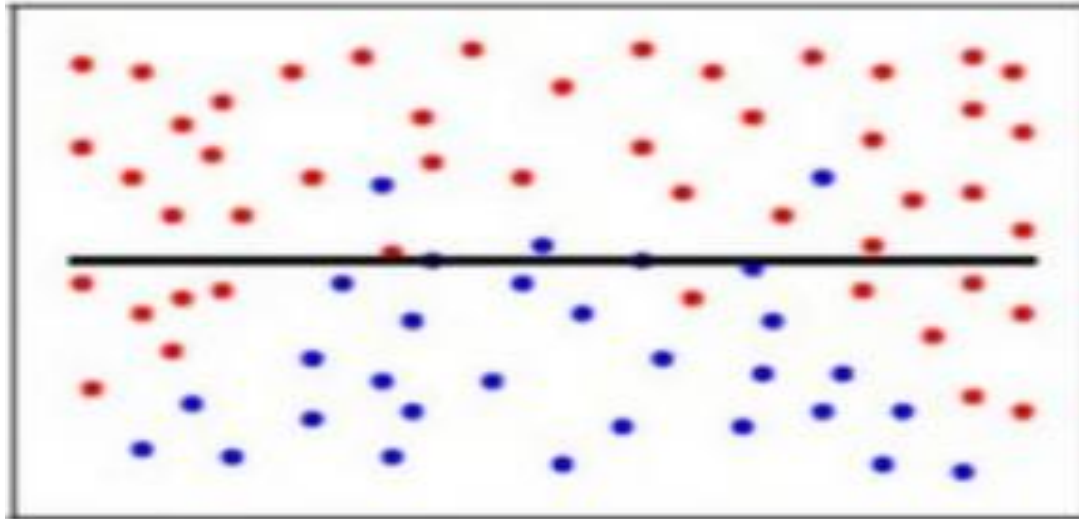
- Bias is the **difference between the Predicted Value and the Expected Value**
- The model makes certain assumptions when it trains on the data provided. When it is introduced to the testing/validation data, these assumptions may not always be correct.
- **Low Bias:** Suggests less assumptions about the form of the target function.
- **High-Bias:** Suggests more assumptions about the form of the target function.
- The inability of a ML algorithm to capture a true relationship is called a bias.



What is Bias?

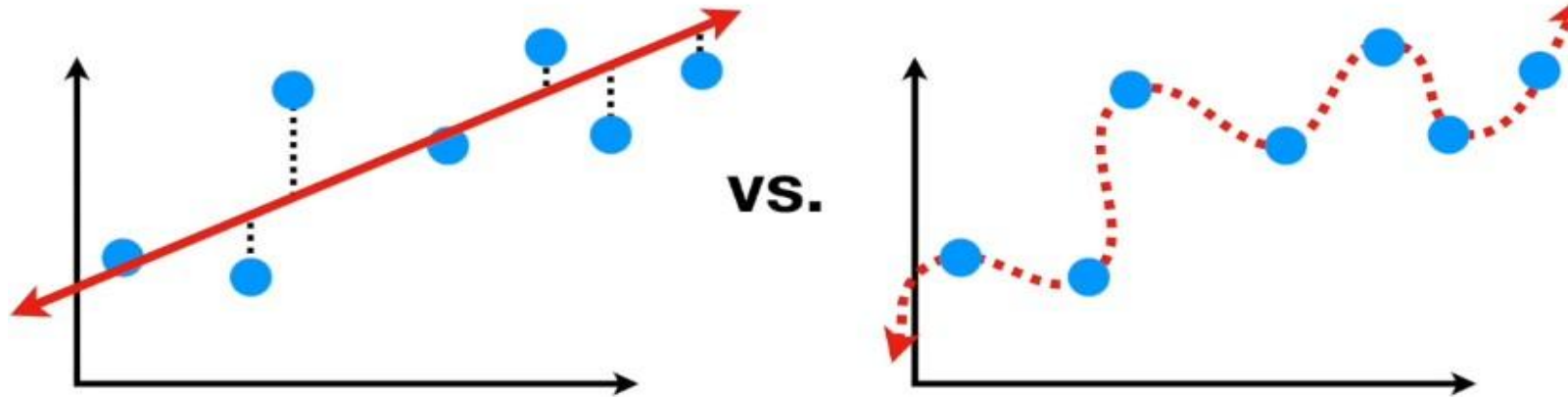
$$\text{Bias}[f'(X)] = E[f'(X) - f(X)]$$

When there is a high bias error, it results in a very simplistic model that does not consider the variations very well. Since it does not learn the training data very well, it is called **Underfitting**.



What is Bias?

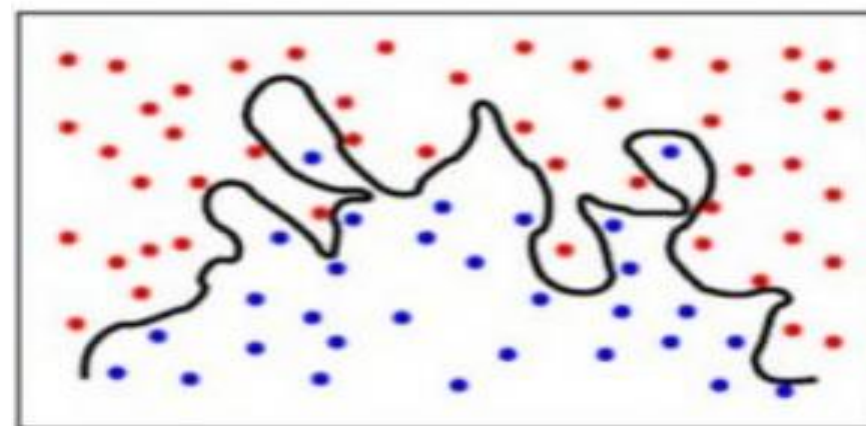
In the contest to see whether the **Straight Line** fits the **training set** better than the **Squiggly Line**...



- Find out the sum of squared distances of the training samples from the fitting line
- **The squiggly line wins** (sum of squared distances =0)

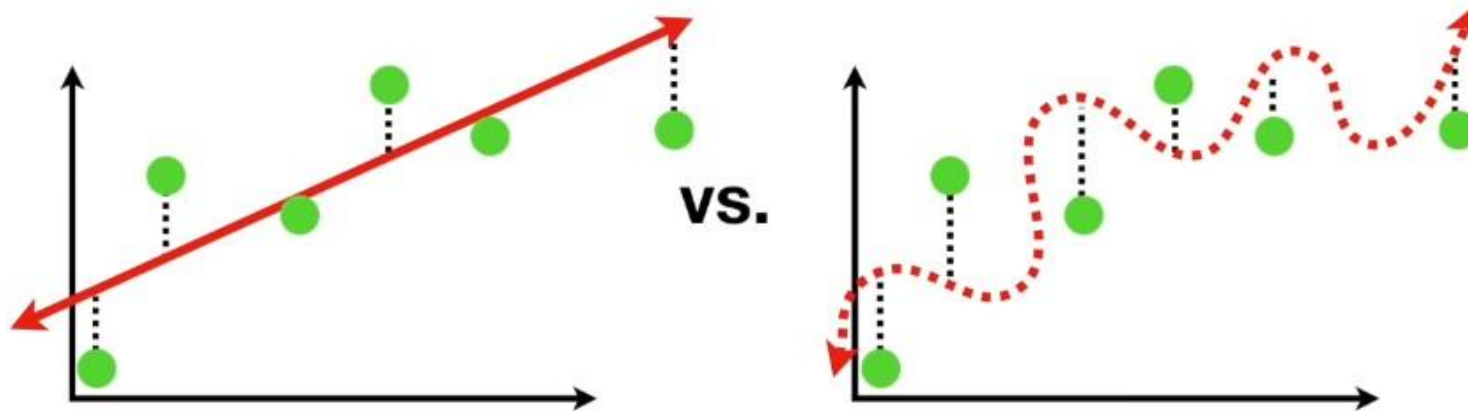
What is a Variance?

- Variance is error introduced in your model due to complex machine learning algorithm, your model learns noise also from the training dataset and performs badly on the test dataset.
- The model with high variance learns too much from the training data, so much so, that when confronted with new (testing) data, it is unable to predict accurately based on it.
- As a result, such models perform very well on training data but has high error rates on test data. It is called **overfitting**.



What is a Variance?

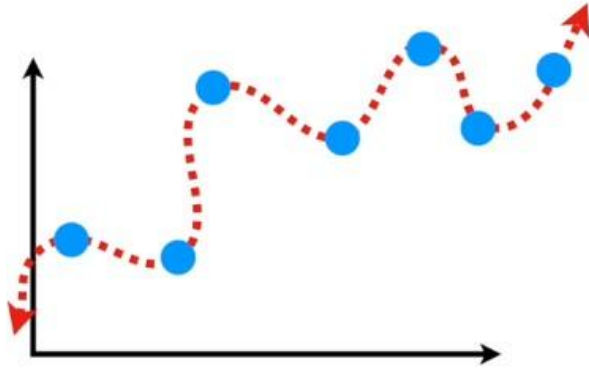
In the contest to see whether the **Straight Line** fits the **testing set** better than the **Squiggly Line**...



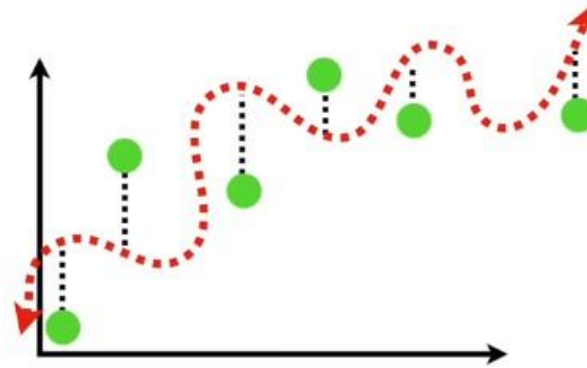
- Find out the sum of squared distances of the testing samples from the fitting line
- **The straight line wins** (sum of squared distances of St. line < squiggly line)

Bias and Variance

In Machine Learning lingo, the difference in fits between data sets is called **Variance**.

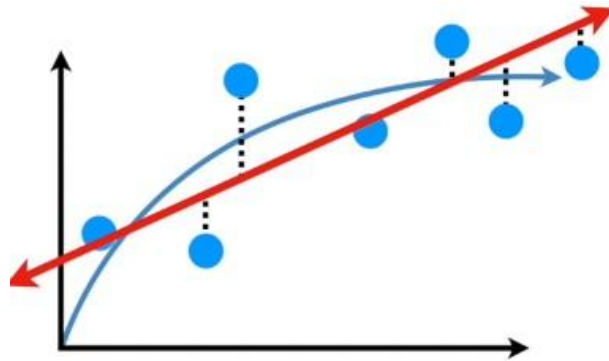


Low Bias

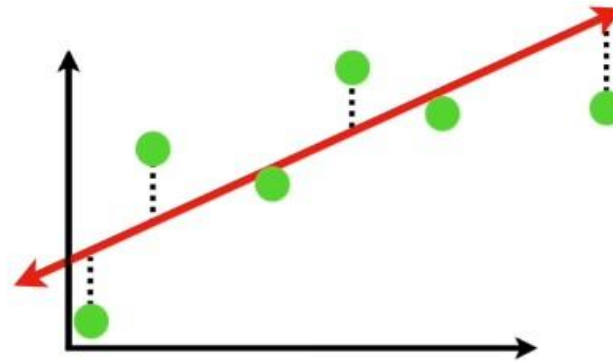


High Variance

Over Fitting



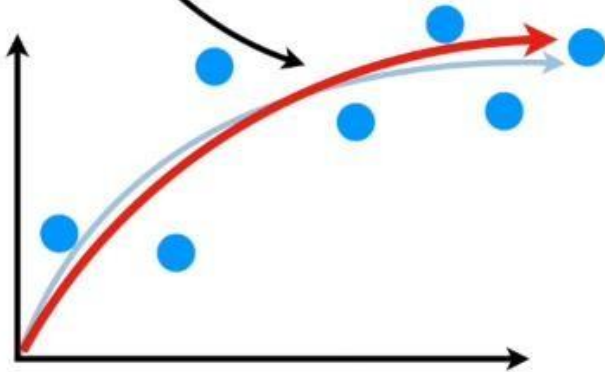
High Bias



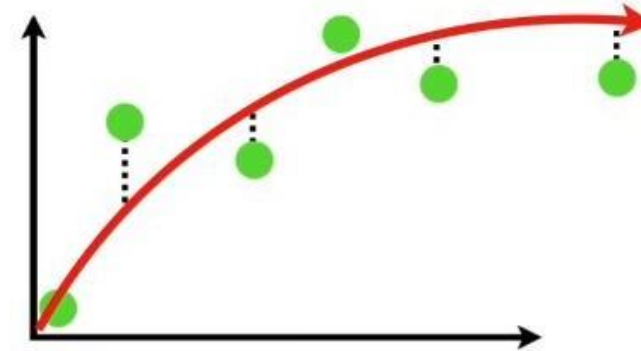
Low Variance

Bias and Variance

In machine learning, the ideal algorithm has **low bias** and can accurately model the true relationship...



...and it has **low variability**, by producing consistent predictions across different datasets.



Find a sweet spot between the simple model and a complex model to have low bias and low variability

Over Fitting & Under Fitting

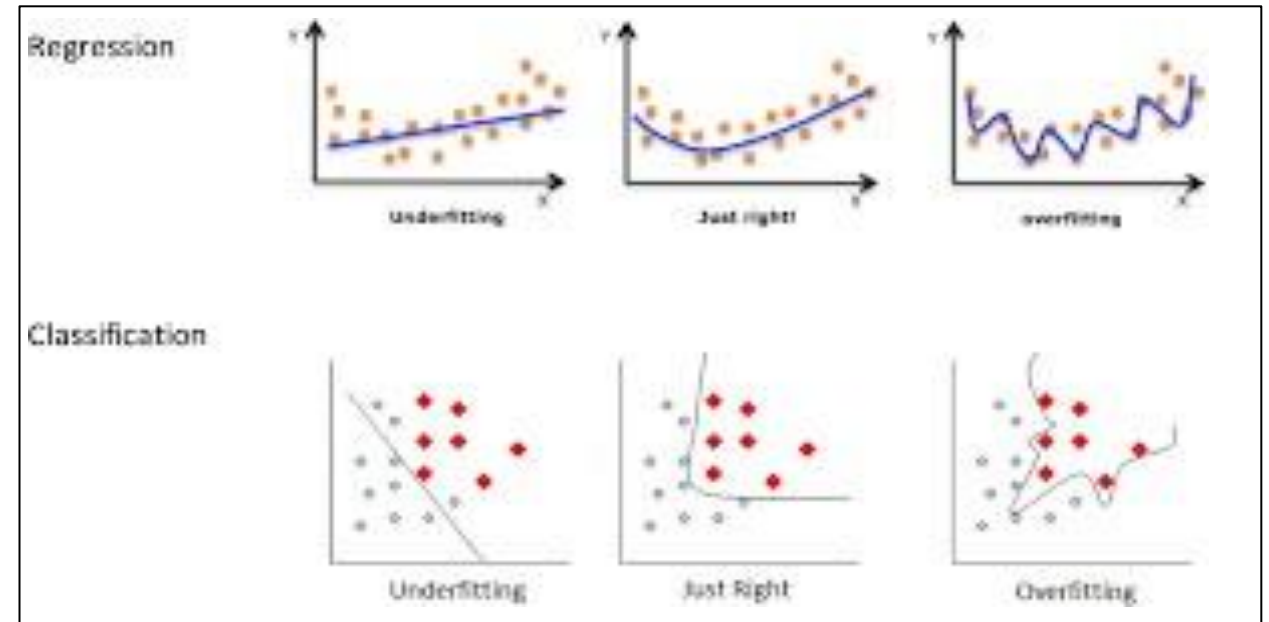
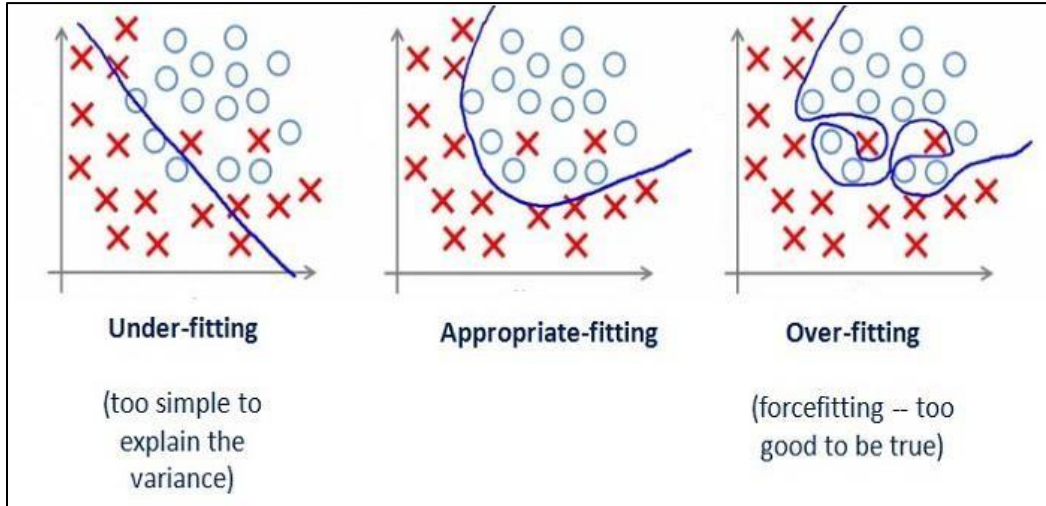
- **Overfitting**

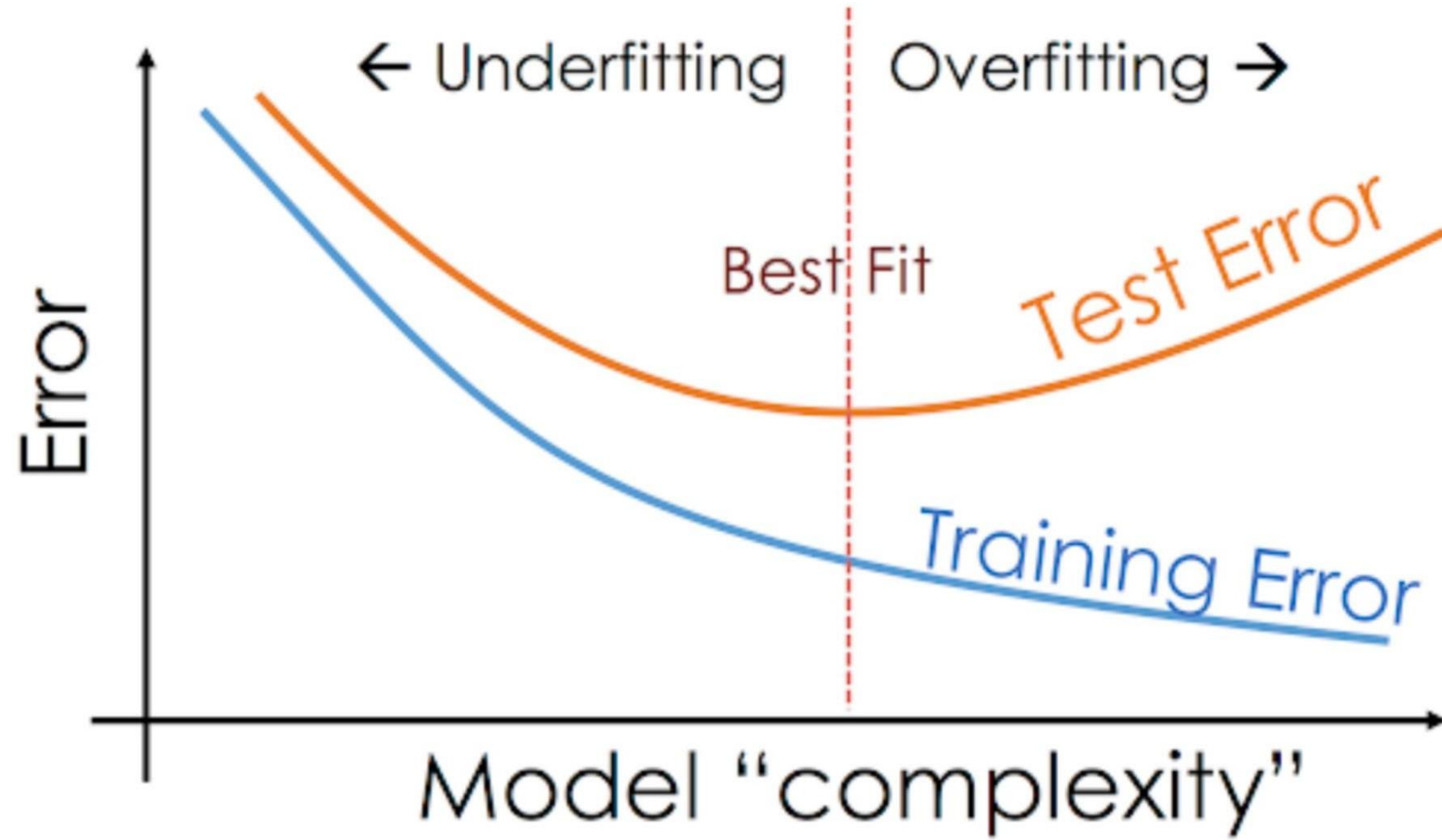
- ✓ occurs when a statistical model or machine learning algorithm captures the noise of the data
- ✓ Intuitively, overfitting occurs when the model or the algorithm fits the data too well
- ✓ Specifically, overfitting occurs if the model or algorithm shows low bias but high variance
- ✓ Overfitting is often a result of an excessively complicated model
- ✓ It happens when we try fitting lots and lots of training data
- ✓ It can be prevented by fitting multiple models and using validation or cross-validation to compare their predictive accuracies on test data.

- **Underfitting**

- ✓ occurs when a statistical model or machine learning algorithm cannot capture the underlying trend of the data
 - ✓ Intuitively, underfitting occurs when the model or the algorithm does not fit the data well enough
 - ✓ Specifically, underfitting occurs if the model or algorithm shows low variance but high bias
 - ✓ Underfitting is often a result of an excessively simple model
 - ✓ It happens when we have very less data for training
- Both overfitting and underfitting lead to poor predictions on new data sets.
 - A model that overfits or underfits is not able to **generalize** well
- Generalization** refers to your model's ability to adapt properly to new, previously unseen data, drawn from the same distribution as the one used to create the model

Over Fitting & Under Fitting





Training, Validation and Testing

Training Dataset

- A training dataset is a dataset of examples used for learning, that is to fit the parameters (e.g., weights) of, for example, a classifier
- Most approaches that search through training data for empirical relationships tend to overfit the data, meaning that they can identify and exploit apparent relationships in the training data that do not hold in general

Source: Wikipedia

Training, Validation and Testing

Validation Dataset

- A validation dataset is a dataset of examples used to tune the Hyperparameter (i.e. the architecture) of a classifier
- It is sometimes also called the development set or the "dev set"
- In artificial neural networks, a hyperparameter is, for example, the number of hidden units.
- It, as well as the testing set (as mentioned above), should follow the same probability distribution as the training dataset.
- In order to avoid overfitting, when any classification parameter needs to be adjusted, it is necessary to have a validation dataset in addition to the training and test datasets.
- For example, if the most suitable classifier for the problem is sought, the training dataset is used to train the candidate algorithms, the validation dataset is used to compare their performances and decide which one to take and, finally, the test dataset is used to obtain the performance characteristics such as accuracy, sensitivity, specificity, F-measure, and so on.
- **The validation dataset functions as a hybrid: it is training data used by testing, but neither as part of the low-level training nor as part of the final testing**

Training, Validation and Testing

Basic Process

- Since our goal is to find the network having the best performance on new data, the simplest approach to the **comparison of different networks** is to evaluate the **error function using data which is independent of that used for training**
- Various networks are trained by minimization of an appropriate error function defined with respect to a training data set.
- The performance of the networks is then compared by evaluating the error function using an independent validation set, and the network having the smallest error with respect to the validation set is selected
- This approach is called the **hold out method**
- Since this procedure can itself lead to some overfitting to the validation set, the performance of the selected network should be confirmed by measuring its performance on a third independent set of data called a test set.

Model Evaluation and Selection

- Evaluation metrics: How can we measure accuracy? Other metrics to consider?
- Use **validation test set** of class-labeled tuples instead of training set when assessing accuracy
- Methods for estimating a classifier's accuracy:
 - Holdout method, random subsampling
 - Cross-validation
 - Bootstrap
- Comparing classifiers:
 - Confidence intervals
 - Cost-benefit analysis and ROC Curves

Evaluation Metrics

- Explain model performance effectively
- Discriminate between different model results
- Ensure robustness before deployment

Common Mistakes

- Analysts often skip robustness checks
- Directly mapping predictions on unseen data is incorrect
- Building a model $\neq$ achieving reliable performance

Classification: Target variable $\rightarrow$ finite categories

Regression: Target variable $\rightarrow$ continuous values

Confusion Matrix

The **confusion matrix** in machine learning is the easiest way to measure the performance of a **classification problem** where the output can be of two or more type of classes.

Prediction outcome

Actual
value

Confusion Matrix

- Predicated output as positive

Prediction outcome	
positive	negative
P	
P	

- Predicated output as negative

Prediction outcome	
positive	negative
P	N
P	N

Confusion Matrix

- Are the values we predicted as positive actually positive in the dataset?
- Are the values we predicted as negative actually negative in the dataset?

		Prediction outcome	
		positive	negative
Actual value	positive	TP	N
	negative	FP	N

		Prediction outcome	
		positive	negative
Actual value	positive	TP	FN
	negative	FP	TN

Confusion Matrix

		Prediction outcome			
		positive	negative		
Actual value	positive	TP	FN	$TP + FN$	Total Actual positive
	negative	FP	TN	$FP + TN$	Total Actual negative

Example

classifications of emails as "spam" or "not spam". Here we are representing class for a spam email as positive (1) and a not spam email as negative (0)

Assign the notations

Actual Classification	0	1	0	1	1	0	0	1	1	1
Predicted Classification	0	1	0	1	0	1	0	0	1	1

Example

Actual Classification	0	1	0	1	1	0	0	1	1	1
Predicted Classification	0	1	0	1	0	1	0	0	1	1
Result	TN	TP	TN	TP	FN	FP	TN	FN	TP	TP

		Prediction outcome	
		positive	negative
Actual value	positive	<i>TP</i>	<i>FN</i>
	negative	<i>FP</i>	<i>TN</i>

Accuracy

It is the ratio of correct predicted values over the total predicted values.

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

		Prediction outcome	
		positive	negative
Actual value	positive	TP 4	FN 2
	negative	FP 1	TN 3

$$\text{Accuracy} = \frac{TP + TN}{TP + FN + FP + TN}$$

$$\text{Accuracy} = \frac{4 + 3}{4 + 1 + 2 + 3} = \frac{7}{10} = 0.7$$

Precision

Precision measures the proportion of true positive instances out of all predicted positive instances. It is calculated as ratio of the number of true positive instances and the sum of true positive and false positive instances.

$$\text{Precision} = \frac{TP}{TP + FP}$$

		Prediction outcome	
		positive	negative
Actual value	positive	TP 4	FN 2
	negative	FP 1	TN 3

$$\text{Precision} = \frac{4}{4 + 1} = \frac{4}{5} = 0.8$$

Recall or Sensitivity

Recall (Sensitivity) is defined as the number of positives classifications by the classifier. Also called **as true positive rate (TPR)**.

$$\text{Recall} = \frac{TP}{TP + FN}$$

		Prediction outcome	
		positive	negative
Actual value	positive	<i>TP</i> 4	<i>FN</i> 2
	negative	<i>FP</i> 1	<i>TN</i> 3

$$\text{Recall} = \frac{4}{4 + 2} = \frac{4}{6} = 0.666$$

False Positive Rate (FPR)

The **false positive rate (FPR)** is the proportion of all actual negatives that were classified *incorrectly* as positives, also known as the **probability of false alarm**. It is mathematically defined as

$$\text{FPR} = \frac{\text{incorrectly classified actual negatives}}{\text{all actual negatives}} = \frac{FP}{FP + TN}$$

False Negative Rate (FNR): It is defined as the fraction of positive examples classified as a negative class by the classifier.

$$\text{FalseNegativeRate} = \frac{\text{FalseNegative}}{\text{TruePositive} + \text{FalseNegative}}$$

Specificity

Specificity, in contrast to recall, is defined as the number of negatives returned by the classifier.

$$\text{Specificity} = \frac{TN}{TN + FP}$$

		Prediction outcome	
		positive	negative
Actual value	positive	TP 4	FN 2
	negative	FP 1	TN 3

$$\text{Specificity} = \frac{3}{3 + 1} = \frac{3}{4} = 0.75$$

F1 Score

F1 score is a balanced measure that takes into account both precision and recall.

$$\text{F1 Score} = 2 \times \frac{(\text{Precision} \times \text{Recall})}{\text{Precision} + \text{Recall}}$$

$$\text{Recall} = \frac{4}{4 + 2} = \frac{4}{6} = 0.666$$

$$\text{Precision} = \frac{4}{4 + 1} = \frac{4}{5} = 0.8$$

$$\text{F1 Score} = 2 \times \frac{(0.8 \times 0.667)}{0.8 + 0.667} = 0.727$$

Precision

Precision measures the proportion of true positive instances out of all predicted positive instances. It is calculated as ratio of the number of true positive instances and the sum of true positive and false positive instances.

$$\text{Precision} = \frac{TP}{TP + FP}$$

		Prediction outcome	
		positive	negative
Actual value	positive	TP 4	FN 2
	negative	FP 1	TN 3

$$\text{Precision} = \frac{4}{4 + 1} = \frac{4}{5} = 0.8$$

Quiz

You need to predict whether a patient has cancer or not. In the training set, the class distribution of the target variable is as follows:

No – 99% and Yes – 1%

In this situation, accuracy will be a better option for evaluating the model.

- A. True
- B. False

Quiz

In confusion matrix, what do true positives represent?

- A. Incorrectly Predicted Positive cases
- B. Correctly Predicted Positive cases
- C. Correctly Predicted Negative cases

Choice of Metric

Metric	Guidance
Accuracy	Use as a rough indicator of model training progress/convergence for balanced datasets. For model performance, use only in combination with other metrics. Avoid for imbalanced datasets. Consider using another metric.
Recall (True positive rate)	Use when false negatives are more expensive than false positives.
False positive rate	Use when false positives are more expensive than false negatives.
Precision	Use when it's very important for positive predictions to be accurate.

Thresholding

ID	Actual Values	Predicted probabilities
ID1	1	0.9
ID2	0	0.51
ID3	1	0.47
ID4	1	0.32
ID5	0	0.1
ID6	1	0.94
ID7	1	0.78
ID8	0	0.56

ID	Actual Values	Predicted probabilities	At 0.5	
ID1	1	0.9	1	TP
ID2	0	0.51	1	FP
ID3	1	0.47	0	FN
ID4	1	0.32	0	FN
ID5	0	0.1	0	TN
ID6	1	0.94	1	TP
ID7	1	0.78	1	TP
ID8	0	0.56	1	FP

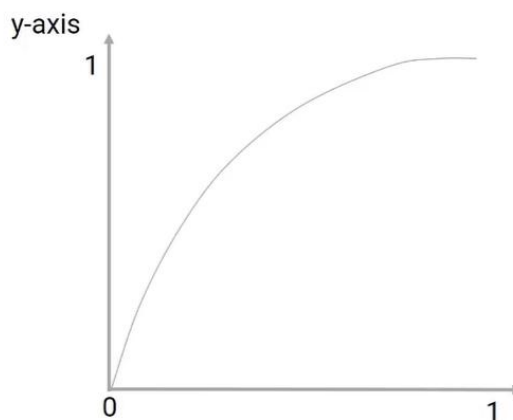
Thresholding

TPR ? FPR?

ID	Actual Values	Predicted probabilities	At 0.5	
ID1	1	0.9	1	TP
ID2	0	0.51	1	FP
ID3	1	0.47	0	FN
ID4	1	0.32	0	FN
ID5	0	0.1	0	TN
ID6	1	0.94	1	TP
ID7	1	0.78	1	TP
ID8	0	0.56	1	FP

Receiver Operating Characteristics

The **ROC curve** is a visual representation of model performance across all thresholds. The long version of the name, receiver operating characteristic, is a holdover from WWII radar detection. The ROC curve is drawn by calculating the true positive rate (TPR) and false positive rate (FPR) at every possible threshold (in practice, at selected intervals), then graphing TPR over FPR.



ROC

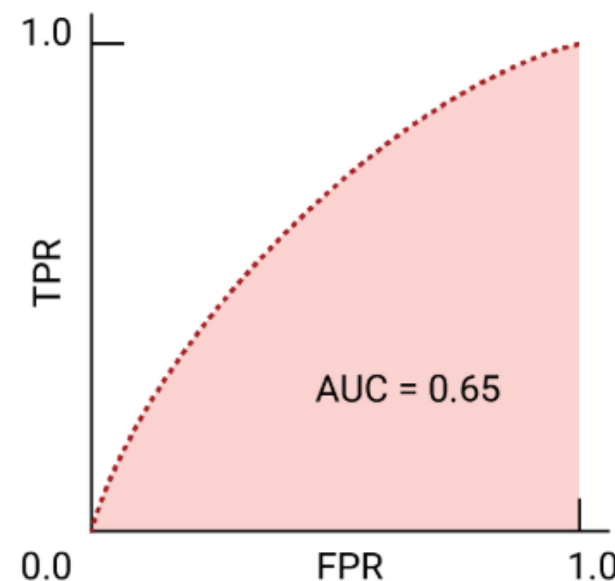
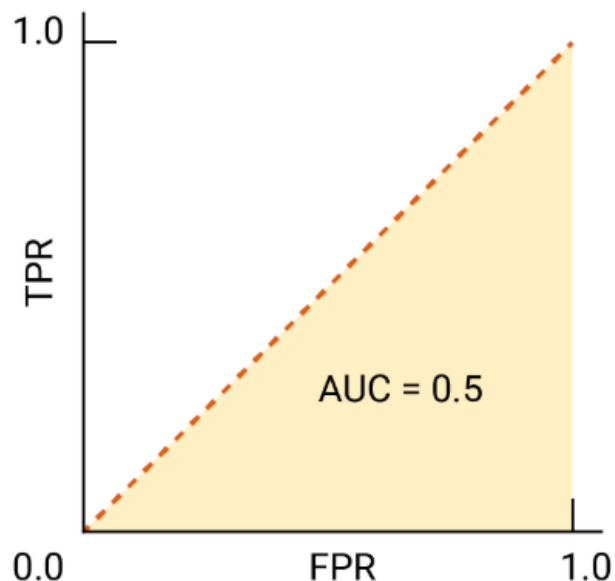
1. Arrange in decreasing order
2. Take the first probability as threshold
3. Calculate TPR, FPR
4. Repeat

ID	Actual Values	Predicted probabilities	At threshold 0.90
ID6	1	0.94	1
ID1	1	0.90	1
ID7	1	0.78	0
ID8	0	0.56	0
ID2	0	0.51	0
ID3	1	0.47	0
ID4	1	0.32	0
ID5	0	0.10	0

AUC

The area under the ROC curve (AUC) represents the probability that the model, if given a randomly chosen positive and negative example, will rank the positive higher than the negative.

The perfect model above, containing a square with sides of length 1, has an area under the curve (AUC) of 1.0



Cross validation

In machine learning, you just can't fit the model to your training data and hope it would accurately work for the real data it has never seen before. ***You need some kind of assurance that your model has got most of the patterns from the data correct, and its not picking up too much on the noise, or in other words its low on bias and variance.***

What is cross validation?

Cross validation is a statistical method used to estimate the performance (or accuracy) of machine learning models.

Non-exhaustive Methods do not compute all ways of splitting the original data.

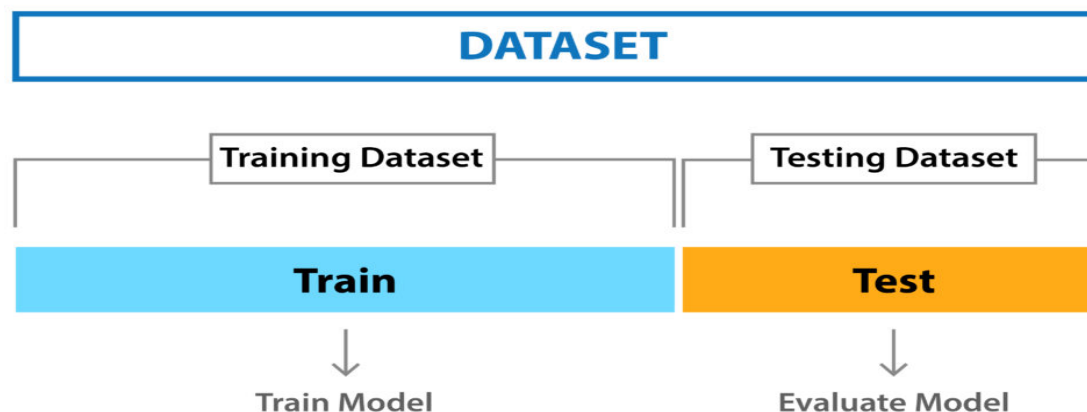
1. Holdout method
2. Random subsampling
3. K-fold Cross-validation

Exhaustive cross validation methods test on all possible ways to divide the original sample into a training and a validation set.

1. Leave-P-Out cross validation
2. Leave-one-out cross validation

Holdout method

- Given a dataset, it is partitioned into two disjoint sets called training set and testing set.
- Classifier is learned based on the training set and get evaluated with testing set.
- If the training set is too large, then model may be good enough, but estimation may be less reliable due to small testing set and vice-versa.



Random Subsampling

- Drawback of Holdout method -It suffers from issues of high variance. This is because it is not certain which data points will end up in the validation set and the result might be entirely different for different sets.
- It is a variation of Holdout method.
- In this method, Holdout method is repeated k times, and in each time, two disjoint sets are chosen at random with a predefined sizes.
- Overall estimation is taken as the average of estimations obtained from each iteration.

K-fold Cross-Validation

- The main *drawback of Random subsampling is, it does not have control over the number of times each tuple is used for training and testing.*
- As there is never enough data to train your model, removing a part of it for validation poses a problem of under fitting. By reducing the training data, we risk losing important patterns/ trends in data set, which in turn increases error induced by bias. So, what we require is a method that provides ample data for training the model and also leaves ample data for validation.
- K-fold Cross-validation is proposed to overcome this problem.

K-fold Cross-Validation

1. Randomly split your entire dataset into k number of folds (subsets)
2. For each fold in your dataset, build your model on $k - 1$ folds of the dataset. Then, test the model to check the effectiveness for kth fold
3. Repeat this until each of the k-folds has served as the test set
4. The average of your k recorded accuracy is called the cross-validation accuracy and will serve as your performance metric for the model.

Iteration 1	Test	Train	Train	Train	Train
Iteration 2	Train	Test	Train	Train	Train
Iteration 3	Train	Train	Test	Train	Train
Iteration 4	Train	Train	Train	Test	Train
Iteration 5	Train	Train	Train	Train	Test

K-fold Cross-Validation

fruit_label	fruit_name
1	Apple
1	Apple
1	Apple
1	Apple
1	Apple
2	Mandarin
...	...
3	Orange
...	...
4	Lemon
4	Lemon
4	Lemon
4	Lemon
4	Lemon

(Folds and dataset shortened for illustration purposes.)

Example has 20 data samples
= 4 classes with 5 samples each.

5-fold CV: 5 folds of 4 samples each.

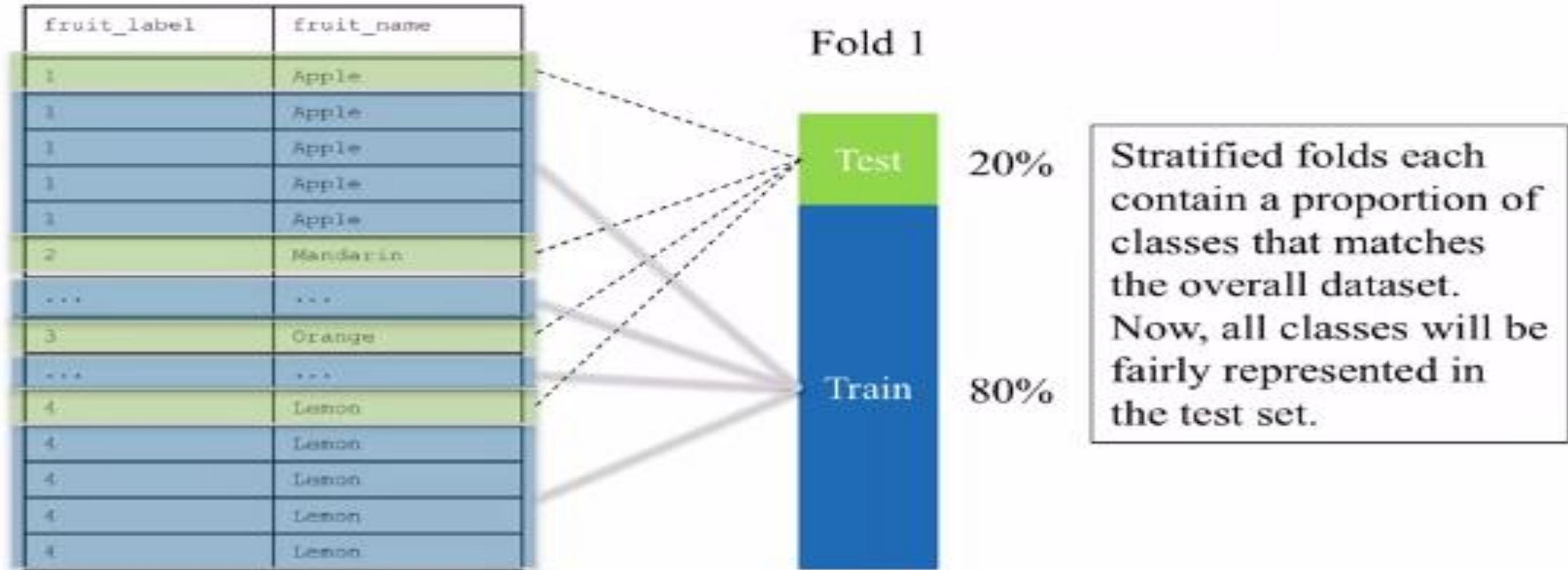
Fold 1 uses the first 20% of the dataset as the test set,
which only contains samples from class 1.

Classes 2, 3, 4 are missing entirely from test set and so
will be missing from the evaluation.

Stratified K Fold Cross Validation

- Random shuffling in K-fold cross validation - chances are we may get highly imbalanced folds which may cause our training to be biased.
- For example, a fold that has majority belonging to one class(say positive) and only a few as negative class- ruins the training
- Stratification is the process of rearranging the data so as to ensure that each fold is a good representative of the whole.
- For example, in a binary classification problem where each class comprises of 50% of the data, it is best to arrange the data such that in every fold, each class comprises of about half the instances.

Stratified K Fold Cross Validation



Leave P-Out Cross Validation

- This approach leaves p data points out of training data, i.e. if there are n data points in the original sample then, ' $n-p$ ' samples are used to train the model and ' p ' points are used as the validation set.
- This is repeated for all combinations in which original sample can be separated this way, and then the error is averaged for all trials, to give overall effectiveness.
- *This method is exhaustive in the sense that it needs to train and validate the model for all possible combinations, and for moderately large p , it can become computationally infeasible.*

Leave one out cross validation.

- Extreme case of K fold cross validation
- Dataset is divided into as many folds as there are instances; thus, all most each tuple forming a training set, building N classifiers.
- In this method, therefore, N classifiers are built from N-1 instances, and each tuple is used to classify a single test instances.
- Test sets are mutually exclusive and effectively cover the entire set (in sequence). This is as if trained by entire data as well as tested by entire data set.
- Overall estimation is then averaged out of the results of N classifiers.
- The leave-one-out cross-validation procedure is appropriate when you have a small dataset or when an accurate estimate of model performance is more important than the computational cost of the method

